

RCP211 - Projet d'Inverse Reinforcement Learning

Fayz El Razaz

21 mai 2026

1 Introduction

L'apprentissage par renforcement (*Reinforcement Learning*, RL) est un domaine de l'intelligence artificielle dans lequel un agent apprend à interagir avec un environnement afin de maximiser une récompense cumulative [7]. À chaque instant, l'agent observe un état de l'environnement, choisit une action, puis reçoit une récompense indiquant la qualité de cette action relativement à l'objectif recherché. Progressivement, l'agent apprend une politique de décision lui permettant d'adopter un comportement performant.

Cette approche a permis d'obtenir des résultats remarquables dans de nombreux domaines tels que les jeux, la robotique ou encore l'optimisation de processus complexes. Les avancées récentes du *Deep Reinforcement Learning* ont notamment permis d'atteindre des performances proches, voire supérieures, à celles de l'humain dans certains environnements complexes [4, 6]. Toutefois, une difficulté majeure du RL réside dans la définition de la fonction de récompense. Dans de nombreux problèmes réels, il est difficile de formaliser explicitement ce qu'est un « bon » comportement. Une récompense mal définie peut conduire l'agent à adopter des stratégies inattendues ou inefficaces.

L'*Inverse Reinforcement Learning* (IRL), également appelé apprentissage inverse par renforcement, propose une approche différente de ce problème [5, 1]. Plutôt que de définir manuellement une fonction de récompense, l'idée est d'observer un comportement expert et d'en déduire la récompense implicite qui semble motiver ce comportement. L'objectif n'est donc plus directement d'apprendre une politique optimale, mais d'apprendre une fonction de récompense cohérente avec les démonstrations observées.

Dans un cadre réel, ces démonstrations peuvent être fournies par un humain réalisant une tâche considérée comme optimale ou experte. Par exemple, dans un contexte robotique, un humain peut guider un bras robotisé afin de montrer le comportement attendu. L'algorithme d'IRL cherche alors à extraire les caractéristiques implicites de ce comportement afin de reconstruire une fonction de récompense permettant ensuite à un agent d'apprendre de manière autonome.

Parmi les approches majeures de l'IRL, les méthodes basées sur le principe de maximum d'entropie occupent une place importante [9, 8]. Ces méthodes considèrent que les démonstrations expertes correspondent à des trajectoires plus susceptibles d'apparaître car elles minimisent implicitement un coût ou maximisent une récompense inconnue. Plus récemment, Finn, Levine et Abbeel ont proposé une extension profonde de cette approche à travers le papier *Guided Cost Learning : Deep Inverse Optimal Control via Policy Optimization* [3], permettant d'apprendre des fonctions de coût complexes à l'aide de réseaux de neurones.

Dans le cadre de ce projet, nous nous intéressons à une version simplifiée de ce problème en utilisant l'environnement **CartPole** de la bibliothèque Gymnasium [2]. Afin de nous concentrer sur les mécanismes fondamentaux de l'IRL sans introduire la complexité d'un système physique réel, les démonstrations expertes ne seront pas produites par un humain mais par un agent

préalablement entraîné à résoudre la tâche de manière performante. Cet agent jouera ainsi le rôle d'un expert dont nous chercherons à reproduire implicitement les objectifs à travers l'apprentissage d'une fonction de récompense.

Bien que notre implémentation soit volontairement simplifiée, les principes fondamentaux étudiés dans le travail de Finn et al. [3] demeurent les mêmes :

- observer un comportement expert ;
- apprendre une récompense cohérente avec ce comportement ;
- puis réentraîner un agent à partir de cette récompense apprise.

L'objectif du projet est donc double :

- comprendre les mécanismes théoriques de l'Inverse Reinforcement Learning ;
- mettre en œuvre une chaîne complète d'apprentissage de récompense sur un environnement simple et contrôlé.

2 Démarche proposée et environnement d'étude

2.1 Choix méthodologique

L'objectif principal de ce projet est de comprendre les mécanismes fondamentaux de l'Inverse Reinforcement Learning dans un cadre suffisamment simple pour permettre une analyse détaillée des différentes étapes de l'apprentissage. Le papier de Finn, Levine et Abbeel propose une approche très avancée appliquée à des systèmes robotiques continus de grande dimension, utilisant des réseaux de neurones profonds et des techniques complexes d'optimisation de politiques. Reprendre l'intégralité de cette architecture dépasserait largement le cadre du projet et introduirait une complexité importante rendant l'analyse pédagogique plus difficile.

Nous avons donc choisi d'adopter une approche simplifiée reposant sur l'environnement `CartPole` de la bibliothèque `Gymnasium`. Ce choix permet de conserver les concepts centraux de l'IRL tout en travaillant dans un environnement :

- simple à visualiser ;
- rapide à entraîner ;
- suffisamment riche pour illustrer les mécanismes de l'apprentissage ;
- largement utilisé comme environnement de référence en apprentissage par renforcement.

L'objectif n'est donc pas ici de reproduire toute la complexité du papier étudié, mais plutôt d'en comprendre les principes fondamentaux à travers une implémentation simplifiée et contrôlée.

2.2 Principe général de la démarche

Dans un problème classique d'apprentissage par renforcement, la fonction de récompense est connue. L'agent interagit avec l'environnement et apprend progressivement une politique lui permettant de maximiser cette récompense.

Dans le cadre de l'Inverse Reinforcement Learning, la situation est inversée :

- nous observons un comportement expert ;
- mais la fonction de récompense qui motive ce comportement est inconnue.

L'objectif devient alors d'apprendre une fonction de récompense cohérente avec les démonstrations observées.

Dans un contexte réel, ces démonstrations seraient généralement fournies par un humain expert réalisant la tâche considérée comme optimale. Par exemple, dans un contexte robotique, un opérateur humain pourrait guider un robot afin de lui montrer le comportement attendu. L'algorithme chercherait ensuite à extraire les caractéristiques implicites de ce comportement afin de reconstruire une fonction de récompense adaptée.

Dans le cadre de ce projet, nous remplaçons cet expert humain par un agent déjà entraîné à résoudre efficacement la tâche. Cette simplification permet de disposer de démonstrations expertes cohérentes et reproductibles tout en conservant exactement la logique théorique de l'IRL.

La démarche générale du projet peut ainsi être résumée en plusieurs étapes :

1. entraîner un agent expert sur l'environnement **CartPole** à l'aide d'un algorithme classique de Reinforcement Learning ;
2. générer plusieurs démonstrations expertes à partir de cet agent ;
3. utiliser ces démonstrations pour apprendre une fonction de récompense ;
4. réentraîner un nouvel agent en utilisant uniquement la récompense apprise ;
5. comparer le comportement obtenu avec celui de l'expert initial.

Cette approche permet d'évaluer si la récompense apprise capture correctement les caractéristiques importantes du comportement expert.

2.3 Présentation de l'environnement CartPole

L'environnement **CartPole** est un problème classique d'apprentissage par renforcement dans lequel un agent doit maintenir un mât vertical en contrôlant un chariot mobile.

Le système est constitué :

- d'un chariot pouvant se déplacer horizontalement ;
- d'un pendule fixé sur ce chariot ;
- d'un agent pouvant appliquer une force vers la gauche ou vers la droite.

L'objectif est de maintenir le pendule en équilibre le plus longtemps possible sans que :

- l'angle du mât ne dépasse une certaine limite ;
- ou que le chariot ne sorte de la zone autorisée.

À chaque instant, l'environnement fournit un état composé de quatre variables :

$$s_t = (x, \dot{x}, \theta, \dot{\theta})$$

où :

- x représente la position horizontale du chariot ;
- $\dot{x}$ représente la vitesse du chariot ;
- θ représente l'angle du pendule ;
- $\dot{\theta}$ représente la vitesse angulaire du pendule.

L'espace d'action est discret :

$$a_t \in \{0, 1\}$$

avec :

- 0 : pousser le chariot vers la gauche ;
- 1 : pousser le chariot vers la droite.

Une trajectoire complète correspond alors à une succession d'états et d'actions :

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$$

Dans le cadre de l'IRL, ces trajectoires expertes constituent les données principales utilisées pour apprendre une fonction de récompense.

2.4 Nature des démonstrations expertes

Les démonstrations expertes correspondent aux trajectoires produites par l’agent préalablement entraîné. Chaque démonstration contient l’ensemble des transitions observées au cours d’un épisode :

$$(s_t, a_t, s_{t+1})$$

ou, de manière plus complète :

$$(s_t, a_t, r_t, s_{t+1}, done_t)$$

Même si la récompense réelle de l’environnement n’est pas utilisée directement pour l’apprentissage inverse, elle peut être conservée à des fins d’analyse ou de validation.

L’idée fondamentale est que l’algorithme d’IRL ne cherche pas à mémoriser directement les actions de l’expert, mais à comprendre quelles caractéristiques des états semblent implicitement valorisées par ce comportement.

Autrement dit, l’objectif n’est pas d’apprendre :

« dans cet état, il faut aller à gauche »

mais plutôt :

« certains états semblent meilleurs que d’autres selon les démonstrations observées »

Cette distinction constitue l’un des éléments centraux de l’Inverse Reinforcement Learning.

3 Construction des démonstrations expertes

3.1 Objectif

Avant de pouvoir apprendre une fonction de récompense par Inverse Reinforcement Learning, il est nécessaire de disposer de démonstrations expertes. Ces démonstrations constituent les exemples de comportements considérés comme performants à partir desquels l’algorithme tentera d’inférer une récompense implicite.

Dans un contexte réel, ces démonstrations pourraient être produites par un humain expert réalisant correctement une tâche donnée. Cependant, afin de simplifier l’expérimentation et de garantir des comportements cohérents et reproductibles, nous utilisons ici un agent préalablement entraîné sur l’environnement **CartPole**. Cet agent jouera le rôle de l’expert dont nous chercherons à reproduire implicitement les objectifs.

3.2 Entraînement d’un agent expert

Pour produire ces démonstrations, nous avons utilisé l’algorithme *Proximal Policy Optimization* (PPO), implémenté dans la bibliothèque **Stable-Baselines3**. PPO est un algorithme moderne d’apprentissage par renforcement particulièrement utilisé pour sa stabilité et ses bonnes performances dans des environnements variés.

Le principe général de PPO est d’apprendre progressivement une politique :

$$\pi(a|s)$$

associant à chaque état s une action a . Dans le cas de **CartPole**, l’agent apprend progressivement à choisir les actions permettant de maintenir le pendule en équilibre le plus longtemps possible.

L’entraînement est réalisé directement sur la récompense native de l’environnement **CartPole-v1**. Une fois l’apprentissage terminé, l’agent atteint des scores élevés proches de la performance maximale autorisée par l’environnement.

Cet agent entraîné constitue alors notre expert.

3.3 Génération des démonstrations

Une fois l’agent expert obtenu, plusieurs épisodes sont générés afin de construire un ensemble de trajectoires expertes.

Chaque épisode correspond à une trajectoire :

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$$

où :

- s_t représente l’état observé à l’instant t ;
- a_t représente l’action choisie par l’expert ;
- T représente la fin de l’épisode.

Dans notre implémentation, chaque transition enregistrée contient :

$$(s_t, a_t, r_t, s_{t+1}, done_t)$$

avec :

- s_t : état courant ;
- a_t : action réalisée ;
- r_t : récompense native de l’environnement ;
- s_{t+1} : état suivant ;
- $done_t$: indicateur de fin d’épisode.

Même si la récompense réelle de l’environnement n’est pas utilisée directement lors de l’apprentissage inverse, elle est conservée afin de faciliter certaines phases d’analyse et de validation expérimentale.

3.4 Filtrage des trajectoires expertes

Toutes les trajectoires produites par l’agent ne sont pas nécessairement conservées. En effet, l’Inverse Reinforcement Learning repose sur l’hypothèse que les démonstrations correspondent à des comportements optimaux ou quasi-optimaux.

Pour cette raison, seules les trajectoires atteignant un score minimal fixé expérimentalement sont conservées dans le dataset final. Dans le cadre de **CartPole**, nous choisissons de ne garder que les épisodes atteignant un score supérieur ou égal à 450.

Ce filtrage permet :

- d’éviter l’introduction de comportements sous-optimaux ;
- de réduire l’ambiguïté lors de l’apprentissage de la récompense ;
- de fournir un ensemble cohérent de démonstrations expertes.

Les trajectoires retenues constituent alors la base de données utilisée dans les étapes suivantes du projet pour apprendre différentes fonctions de récompense.

3.5 Construction du dataset final

Les trajectoires expertes retenues sont finalement regroupées dans un dataset contenant l’ensemble des transitions observées lors des épisodes experts.

Ce dataset contient notamment :

- les états ;
- les actions ;
- les états suivants ;
- les indicateurs de fin d’épisode.

Ces données serviront ensuite à entraîner différents modèles de récompense dans le cadre de l’Inverse Reinforcement Learning.

L’objectif sera alors de construire une fonction :

$$r_{\theta}(s, a)$$

capable d’attribuer des récompenses élevées aux comportements proches des démonstrations expertes et des récompenses plus faibles aux comportements non experts.

4 Modélisation de la fonction de récompense

Nous disposons désormais d’un ensemble de démonstrations expertes construites à partir d’un agent PPO préalablement entraîné sur l’environnement `CartPole`. Ces démonstrations constituent la base de travail permettant d’aborder le véritable problème de l’Inverse Reinforcement Learning : apprendre une fonction de récompense cohérente avec le comportement observé.

L’objectif n’est plus ici d’apprendre directement une politique, mais de construire une fonction :

$$r_{\theta}(s, a)$$

capable d’attribuer des récompenses élevées aux comportements proches de ceux de l’expert et des récompenses plus faibles aux comportements non experts.

Autrement dit, nous cherchons à reconstruire implicitement ce qui semble être valorisé par l’expert à travers les trajectoires observées. Une fois cette récompense apprise, elle pourra ensuite être utilisée comme signal d’apprentissage pour entraîner un nouvel agent par Reinforcement Learning classique.

Cette étape constitue le cœur du projet, car elle correspond au passage :

comportement expert $\rightarrow$ fonction de récompense

alors qu’en Reinforcement Learning classique, le problème est généralement :

fonction de récompense $\rightarrow$ politique optimale.

4.1 Principe général

Le principe retenu dans ce projet consiste à entraîner un modèle paramétrique capable de différencier :

- les états provenant des démonstrations expertes ;
- les états provenant de comportements non experts.

L’idée est que si un état apparaît fréquemment dans les trajectoires expertes, alors cet état doit probablement être considéré comme désirable par la fonction de récompense recherchée.

Le modèle appris attribuera donc progressivement :

- des scores élevés aux états experts ;
- des scores plus faibles aux états aléatoires ou sous-optimaux.

Dans une étape ultérieure, cette fonction de récompense sera utilisée comme signal d’apprentissage dans un nouvel entraînement par Reinforcement Learning afin d’évaluer si elle permet effectivement de reconstruire un comportement proche de celui de l’expert initial.

4.2 Difficulté de la modélisation

L'apprentissage d'une fonction de récompense constitue cependant un problème difficile. Contrairement à un problème supervisé classique, il n'existe pas nécessairement une unique récompense expliquant le comportement observé.

Dans le cas de `CartPole`, plusieurs formulations peuvent conduire à un comportement performant :

- maintenir le pendule proche de la verticale ;
- limiter les vitesses angulaires ;
- éviter les états de chute ;
- maximiser simplement la durée de survie de l'épisode.

Toutes ces récompenses peuvent potentiellement produire des politiques proches. Le problème est donc sous-déterminé : plusieurs fonctions de récompense différentes peuvent expliquer les mêmes démonstrations expertes.

Pour cette raison, l'objectif n'est pas nécessairement de retrouver exactement la récompense native de l'environnement, mais plutôt de construire une récompense permettant de reproduire un comportement similaire à celui de l'expert.

4.3 Modèles étudiés

Afin d'étudier l'impact de la modélisation de la récompense sur les performances finales, plusieurs approches seront testées et comparées.

Nous commencerons par des modèles simples et interprétables, avant d'utiliser des modèles plus expressifs :

- une récompense linéaire ;
- une récompense quadratique inspirée des propriétés physiques du problème ;
- une récompense basée sur un réseau de neurones.

Cette progression permettra :

- d'étudier la capacité de chaque modèle à capturer les comportements experts ;
- d'observer les limites des modèles simples ;
- d'évaluer l'intérêt des représentations plus complexes.

Les différentes récompenses apprises seront ensuite utilisées pour entraîner de nouveaux agents, dont les performances seront comparées à celles de l'expert initial.

Références

- [1] Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the 21st International Conference on Machine Learning (ICML)*, 2004.
- [2] Farama Foundation. Gymnasium documentation, 2023. <https://gymnasium.farama.org/>.
- [3] Chelsea Finn, Sergey Levine, and Pieter Abbeel. Guided cost learning : Deep inverse optimal control via policy optimization. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.
- [4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540) :529–533, 2015.
- [5] Andrew Y. Ng and Stuart Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the 17th International Conference on Machine Learning (ICML)*, 2000.
- [6] David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587) :484–489, 2016.

- [7] Richard S. Sutton and Andrew G. Barto. Reinforcement learning : An introduction. *MIT Press*, 2018.
- [8] Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.
- [9] Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2008.